Mastering Terraform Remote Backends Step by Step

Terraform state tracks your real-world cloud resources, making it critical to store safely. Storing state remotely prevents data loss, enables team collaboration, and stops conflicting deployment mistakes.

What you'll learn:

1. 1. Configuring an AWS S3 Backend
2. 2. Adding State Locking with DynamoDB
3. 3. Migrating State from Local to Remote
4. 4. Enabling Encryption and Versioning

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Configuring an AWS S3 Backend

To move your state off your laptop, define a backend block inside your Terraform configuration. Point it to an AWS S3 bucket where your file will be saved remotely. This ensures your teammates always read the exact same infrastructure state as you.

```
terraform {
  backend "s3" {
    bucket = "my-company-tf-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
  }
}
```

2

TIP 2 OF 4

2. Adding State Locking with DynamoDB

If two engineers run terraform apply at once, state file corruption will happen. Adding DynamoDB table locking forces Terraform to lock state during execution. Everyone else must wait until your deployment finishes successfully.

```
terraform {
  backend "s3" {
    bucket         = "my-company-tf-state"
    key            = "prod/tf.tfstate"
    region         = "us-east-1"
    dynamodb_table = "tf-state-locks"
  }
}
```

Terraform Backends: Store State Remotely and Safely

Friday, October 30, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Migrating State from Local to Remote

When you add a backend block to existing local Terraform code, you must re-initialize Terraform. Terraform detects the backend change and offers to copy your local state file into the remote S3 bucket seamlessly.

```
$terraform init -migrate-state
```

4

TIP 4 OF 4

4. Enabling Encryption and Versioning

Terraform state files contain raw sensitive values like database credentials. Enable bucket versioning on S3 to recover older state versions if needed, and set encrypt to true so data is encrypted at rest.

```
terraform {
  backend "s3" {
    bucket  = "my-company-tf-state"
    key     = "prod/tf.tfstate"
    region  = "us-east-1"
    encrypt = true
  }
}
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always enable object versioning on your S3 state bucket to restore state if corrupted.
+ Use separate state keys or buckets for staging and production environments.
+ Never commit your .tfstate or .tfstate.backup files to Git version control.
+ Use DynamoDB state locking to stop team members from overwriting concurrent changes.
+ Restrict access to the remote backend bucket using IAM policies with least privilege.

Watch Out For

! Hardcoding AWS access credentials directly inside the backend configuration block.
! Forgetting to enable S3 encryption when your state contains secrets or passwords.
! Manually modifying the JSON inside terraform.tfstate directly instead of using terraform state commands.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh